

Лекция 11

Стандартная библиотека шаблонов

1 Содержимое STL

STL – стандартная библиотека шаблонов. Ядро библиотеки образуют три элемента: контейнеры, алгоритмы и итераторы.

Контейнеры (containers) - это объекты, предназначенные для хранения набора элементов. Например, вектор, линейный список, множество. Ассоциативные контейнеры (associative containers) позволяют с помощью ключей получить быстрый доступ к хранящимся в них значениям.

Алгоритмы (algorithms) выполняют операции над содержимым контейнера. Существуют алгоритмы для инициализации, сортировки, поиска, замены содержимого контейнеров.

Итераторы (iterators) - это объекты, которые по отношению к контейнеру играют роль указателей. Они позволяют получить доступ к содержимому контейнера и сканировать его элементы, примерно так же, как указатели используются для доступа к элементам массива. С итераторами можно работать так же, как с указателями. К ним можно применять операции *, инкремент, декремент.

2 Классы-контейнеры в STL

Наименование класса	Описание	Заголовочный файл
bitset	множество битов	<code><bitset></code>
vector	динамический массив	<code><vector></code>
list	линейный список	<code><list></code>
deque	двусторонняя очередь	<code><deque></code>
stack	стек	<code><stack></code>
queue	очередь	<code><queue></code>
priority_queue	очередь с приоритетом	<code><queue></code>
map	ассоциативный список для хранения пар ключ/значение, где с каждым ключом связано одно значение	<code><map></code>
multimap	с каждым ключом связано два или более значений	<code><map></code>
set	множество	<code><set></code>
multiset	множество, в котором каждый элемент не обязательно уникален	<code><set></code>

Методы для включения и исключения элементов

Метод	Описание
<code>insert(p, x)</code>	добавление x перед элементом, на который указывает p
<code>insert(p, n, x)</code>	добавление n копий x перед p
<code>insert(p, first, last)</code>	добавление элементов из диапазона [first:last] перед p
<code>push_back(x)</code>	добавление x в конец
<code>push_front(x)</code>	добавление нового первого элемента (только для списков и очередей с двумя концами)
<code>pop_back()</code>	удаление последнего элемента
<code>pop_front()</code>	удаление первого элемента (только для списков и очередей с двумя концами)
<code>erase(p)</code>	удаление элемента в позиции p
<code>erase(first, last)</code>	удаление элементов из диапазона [first:last]
<code>clear()</code>	удаление всех элементов

Операции STL

Операция	Описание
size()	количество элементов
empty()	определяет, пуст ли контейнер
capacity()	память, выделенная под вектор (только для векторов)
reserve(n)	выделяет память для контейнера под n элементов
resize(n)	изменяет размер контейнера (только для векторов, списков и очередей с двумя концами)
swap(x)	обмен местами двух контейнеров
==, !=, <	операции сравнения
operator =(x)	контейнеру присваиваются элементы контейнера x
assign(n, x)	присваивание контейнеру n копий элементов x (не для ассоциативных контейнеров)
assign(first, last)	присваивание элементов из диапазона [first:last]
operator [](k)	доступ к элементу с ключом k
find(k)	находит элемент с ключом k
lower_bound(k)	находит первый элемент с ключом, меньшим k
upper_bound(k)	находит первый элемент с ключом, большим k
equal_range(k)	находит lower_bound (нижнюю границу) и upper_bound (верхнюю границу) элементов с ключом k

3 Итераторы в STL

Синтаксис объявления итератора

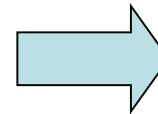
```
<контейнер> <тип> :: iterator <имя_итератора>;
```

Методы итераторов

Метод	Описание
begin()	указывает на первый элемент
end()	указывает на элемент, следующий за последним

4 Примеры

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int main()
7  {
8      // создание пустого вектора типа int
9      vector<int> myVector;
10
11     // вставка элементов в конец вектора
12     myVector.push_back(10);
13     myVector.push_back(30);
14     myVector.push_back(50);
15
16     // печать значений элементов вектора
17     for(int i = 0; i < myVector.size(); i++)
18     { cout << myVector[i] << ' '; }
19
20     return 0;
21 }
```

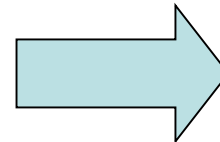


10 30 50

```

1  #include <iostream>
2  #include <vector>
3  #include <string>
4  using namespace std;
5
6  class People
7  {
8      string Name;
9      int Age;
10 public:
11     People():Name("Alex"), Age(20){}
12     People(string N, int A):Name(N), Age(A){}
13     void Show()
14     { cout << Name << ", " << Age << endl; }
15 };
16
17 int main()
18 {
19     // создание пустого вектора
20     vector<People> myVector;
21
22     // вставка элементов в конец вектора
23     myVector.push_back(People());
24     myVector.push_back(People("Bob", 10));
25     myVector.push_back(People("Ann", 30));
26
27     // печать значений элементов вектора
28     for(int i = 0; i < myVector.size(); i++)
29     { myVector[i].Show(); }
30 }

```



```

Alex, 20
Bob, 10
Ann, 30

```



```
1 #include <iostream>
2 #include <list>
3
4 using namespace std;
5
6 int main()
7 {
8     // создание пустого списка типа int
9     list<int> myList;
10
11     // вставка элементов в начало списка
12     myList.push_front(10);
13     myList.push_front(30);
14     myList.push_front(50);
15
16     // создание итератора
17     list<int> :: iterator it;
18     // печать значений элементов списка
19     for (it = myList.begin(); it != myList.end(); it++)
20     { cout << (*it) << " "; }
21
22     return 0;
23 }
```



50 30 10

```

1  #include <iostream>
2  #include <list>
3  #include <string>
4  using namespace std;
5
6  class People
7  {
8      string Name;
9      int Age;
10 public:
11     People():Name("Alex"), Age(20){}
12     People(string N, int A):Name(N), Age(A){}
13     void Show()
14     { cout << Name << ", " << Age << endl; }
15 };
16
17 int main()
18 {
19     // создание пустого списка
20     list<People> myList;
21
22     // вставка элементов в начало списка
23     myList.push_front(People());
24     myList.push_front(People("Bob", 10));
25     myList.push_front(People("Ann", 30));
26
27     // создание итератора
28     list<People> :: iterator it;
29     // печать значений элементов списка
30     for (it = myList.begin(); it != myList.end(); it++)
31     { it->Show(); }
32 }

```



```

Ann, 30
Bob, 10
Alex, 20

```